RAG-Based Architecture Overview (Text Flow Model)

Query Input (User)
$\Downarrow$ Natural Language Processing
LLM Encoder
$\Downarrow$ Convert Query to Embedding Vector q
Vector Similarity Search (FAISS / Chroma / Pinecone)
$\Downarrow$ Retrieve Top-K Docs Based on Similarity Score
Knowledge Store/Database/Corpus
$\Downarrow$ Relevant Knowledge Chunks $\{d_1, d_2, ..., d_k\}$
LLM Reasoning + Response Generation
$\Downarrow$ Final Answer With Grounded Evidence

Core Flow: User $\rightarrow$ Embedding $\rightarrow$ Retrieval $\rightarrow$ LLM $\rightarrow$ Answer

RAG Mathematical Foundations

1. Embedding Representation Text x maps to vector $e_x \in \mathbb{R}^n$:

$$e_x = [e_1, e_2, ..., e_n]$$

2. Similarity-Based Retrieval Cosine similarity for ranking d^*:

$$\text{sim}(q, d) = \frac{e_q \cdot e_d}{\|e_q\| \cdot \|e_d\|}, \quad d^* = \arg\max_d \text{sim}$$

3. Probabilistic Retrieval Softmax over Euclidean distance:

$$P(d_i|q) = \frac{\exp\left(-\frac{\|e_q - e_i\|^2}{2\sigma^2}\right)}{\sum_j \exp\left(-\frac{\|e_q - e_j\|^2}{2\sigma^2}\right)}$$

4. LLM Conditional Generation Token-wise probability:

$$P(y|q, D) = \prod_t P(y_t|y_{<t}, q, D)$$

5. Fusion via Attention

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Summary of Operations

Process	Math Concept
Embedding	Vectors in $\mathbb{R}^n$
Retrieval	Cosine / Distance
Ranking	Softmax Prob.
Generation	Conditionals
Fusion	Attention Weights

Implementation: Vector Space & Similarity

1. Temperature Tuning (LLM Generation)

```python
def __init__(self, vector_store: VectorStoreService, session_manager: SessionManager):
    self.vector_store = vector_store
    self.session_manager = session_manager
    # Using Gemini 2.5 Flash — faster and cheaper, suitable for interactive RAG
    self.llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.5)
    logger.info("Q&A Agent (Gemini) initialized")

async def answer_question(self, session_id: str, question: str) -> Dict:
    """
    Answer question using RAG with conversation history
    """
    try:
        # Retrieve relevant documents
        relevant_docs = self.vector_store.similarity_search(
            session_id, question, k=5
        )
```

Implementation of Temperature Tuning

2. Markov Model

```python
def markov_rag_generate(query: str,
                        retrieved_docs: List[str],
                        steps=60,
                        temperature=0.5):

    # 1) Convert retrieved docs → embedding-based influence vector
    context_text = " ".join(retrieved_docs)
    tokens = tokenizer(context_text, return_tensors="pt").input_ids[0]

    # Convert context to influence weights (simple version → average attention mass)
    with torch.no_grad():
        emb = model(tokens).logits.mean(dim=0)
        context_embedding = torch.softmax(emb, dim=-1)

    # 2) Start sequence with query prompt
    input_ids = tokenizer.encode(query, return_tensors="pt")[0]
    prev_token_id = input_ids[-1]

    generated = [prev_token_id]

    # 3) Markov sampling loop → P(x_t | x_{t-1})
    for _ in range(steps):
        next_id = markov_next_token(prev_token_id, context_embedding, temperature)
        generated.append(next_id)
        prev_token_id = next_id

    text = tokenizer.decode(generated, skip_special_tokens=True)
    return text.strip()

```

Use of Markov Model to use the full model accurately

Implementation: Probabilistic Retrieval

3. Gaussian Kernel (RBF)

```python
def compute_retrieval_probs(
    q_emb: np.ndarray,
    doc_embs: np.ndarray,
    sigma: float = 1.0,
    eps: float = 1e-12
) -> np.ndarray:

    if q_emb.ndim != 1:
        q_emb = q_emb.ravel()
    # squared Euclidean distances
    # careful: compute in a numerically stable vectorized way
    diffs = doc_embs - q_emb  # shape (N, D)
    sq_dists = np.sum(diffs * diffs, axis=1)  # shape (N,)

    # kernel exponent: -sq_dist / (2 * sigma^2)
    denom = 2.0 * (sigma ** 2)
    exps = np.exp(-sq_dists / (denom + eps))

    # normalize to probabilities
    sump = np.sum(exps)
    if sump <= 0.0:
        # fallback: uniform
        probs = np.ones_like(exps) / len(exps)
    else:
        probs = exps / sump
    return probs
```

Softmax Scaling via Kernel

4. Cosine Similarity

```python
class VectorStoreService:
    def similarity_search(self, session_id: str, query: str, k: int = 5) -> List[Document]:
        """Perform a similarity search."""
        try:
            collection_name = f"session_{session_id}"

            if collection_name not in self.collections:
                vectorstore = Chroma(
                    collection_name=collection_name,
                    embedding_function=self.embeddings,
                    persist_directory=self.persist_directory,
                    client=self.client,
                )
                self.collections[collection_name] = vectorstore
            else:
                vectorstore = self.collections[collection_name]

            results = vectorstore.similarity_search(query, k=k)
            logger.info(f" Found {len(results)} similar docs for query.")
            return results

        except Exception as e:
            logger.error(f"Error in similarity search: {str(e)}")
            return []

    def similarity_search_with_score(
        self, session_id: str, query: str, k: int = 5
    ) -> List[tuple]:
        """Search for similar documents with relevance scores."""
```

use of Cosine Similarity